

RAP Cloud 課程 3：Determination / Validation + 完整 strict(2)

Lecture

這一課要證明的事

rc02 刻意用了「零 Implementation Class」的最簡 Managed BDEF，把「Managed CUD 真的能執行完」這件事單獨驗證清楚，**strict(2)** 與 Implementation Class 留到這一課一起做。這一課要做三件事：

1. 補齊完整的 **strict(2)** (含 **authorization master**)，驗證這個 Cloud 環境真的支援官方文件描述的完整檢查等級
2. 用 **Determination** (**on save**，自動填入 **created_at/created_by**) 跟 **Validation** (**on save**，擋掉非法的 **status** 值) 示範 Managed RAP BO 最核心的兩種宣告式擴充機制
3. 對照舊課程 rap05 / rap06——那兩課因為 On-Premise 系統語法版本落後，被迫用官方文件標成「obsolete」的 **FOR DETERMINATION / FOR VALIDATION** 語法；這一課要驗證這個 Cloud 環境能不能直接採用官方現行語法 **FOR DETERMINE ON SAVE / FOR VALIDATE ON SAVE**

結論先講：官方現行語法完全可以用，不需要任何 obsolete 轉換——這是這門課到目前為止第一次「新舊語法落差」完全不存在的一課。但過程中踩到三個這個環境特有的坑，其中最後一個 (**ROLLBACK ENTITIES**) 是官方文件明確記載、但極容易被忽略的 RAP 事務語意，值得花時間搞懂。

完整 strict(2) 版本 BDEF

```
managed implementation in class zbp_i_rc01_task unique;
strict ( 2 );

define behavior for ZI_RC01_TASK alias Task
persistent table zrc01_task
lock master
authorization master ( none )
etag master created_at
{
    create;
    update;
    delete;

    field ( readonly : update ) task_id;
    field ( readonly )          created_at, created_by;
    field ( mandatory )         description;

    determination setCreationInfo on save { create; }
    validation validateStatus on save { field status; }
}
```

跟 rc02 相比多了三塊：`managed implementation in class ...` (宣告 Implementation Class)、`authorization master(none)` (滿足 `strict(2)` 要求宣告 `authorization` 的硬性規定)、`determination/validation` 兩行宣告。

`authorization master(none)`：用一行滿足 `strict(2)`，同時完全不用碰權限實作

rc02 已經用真實錯誤訊息追出因果鏈：`strict(2)` 強制要求宣告 `authorization`，宣告 `authorization master` 才連帶要求 Implementation Class。這一課既然本來就要有 Implementation Class (因為要寫 Determination/Validation)，`authorization` 這關無論如何躲不掉，問題變成：要不要連權限檢查邏輯也一起寫？

查證官方文件 `ABENBDL_AUTHORIZATION`：`authorization master` 有四種寫法——`(global)` / `(instance)` / `(global, instance)` / `(none)`。`(none)` 明文規定「不能替這個值實作任何 **RAP 權限 Handler Method**」 ("When none is specified, it is not possible to implement RAP handler methods for authorizations")——也就是說，用 `(none)` 相當於明確宣告「這個 RAP BO 完全不做權限檢查」，換來的好處是 Implementation Class 完全不用碰權限那一塊，可以專心處理這一課真正的主題 (Determination / Validation)。

這是這一課的教學選擇，不代表 `(none)` 是正式產品該用的值——真實專案的 RAP BO 幾乎一定要做權限檢查 (`(global)` / `(instance)` 或兩者疊加)，這一課只是先隔離變因，之後如果要開權限相關的延伸內容，再回頭示範 `(global)/(instance)` 該怎麼實作。

Handler Method 不能直接寫在 `FOR BEHAVIOR OF` 的全域類別——要用 Local Types 裡的 `lhc_task`

第一次嘗試把 Determination/Validation 的 Handler Method 直接宣告在 `ZBP_I_RC01_TASK` (`managed implementation in class` 指名的那個全域類別) 本體時，Eclipse 給出精確的錯誤訊息：

```
The local class "ZBP_I_RC01_TASK" must be derived from
"CL_ABAP_BEHAVIOR_HANDLER" to define BEHAVIOR methods.
```

正確結構分兩層：

- 全域類別 (`FOR BEHAVIOR OF` 指名的那個，Eclipse 類別編輯器上方主要區域) 維持空殼，什麼都不寫：

```
``abap CLASS zbp_i_rc01_task DEFINITION PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF zi_rc01_task.
ENDCLASS.
```

```
CLASS zbp_i_rc01_task IMPLEMENTATION. ENDCLASS. ``
```

- 真正的 **Handler 邏輯** 寫在一個繼承自 `CL_ABAP_BEHAVIOR_HANDLER` 的 **Local Class** (慣例命名 `lhc_<entity別名>`)，放在 Eclipse 類別編輯器下方的「**Local Types**」分頁 (不是「Class-relevant Local Types」，是「Local Types」)：

```
``abap CLASS lhc_task DEFINITION INHERITING FROM cl_abap_behavior_handler. PRIVATE SECTION.
METHODS setCreationInfo FOR DETERMINE ON SAVE IMPORTING keys FOR Task~setCreationInfo. METHODS
validateStatus FOR VALIDATE ON SAVE IMPORTING keys FOR Task~validateStatus. ENDCLASS.
```

```
CLASS lhc_task IMPLEMENTATION.
```

```
METHOD setCreationInfo. MODIFY ENTITIES OF zi_rc01_task IN LOCAL MODE ENTITY Task UPDATE FIELDS (
created_at created_by ) WITH VALUE #( FOR key IN keys ( %key = key-%key created_at = utclong_current( )
created_by = cl_abap_context_info=>get_user_technical_name( ) ) ). ENDMETHOD.
```

```
METHOD validateStatus. READ ENTITIES OF zi_rc01_task IN LOCAL MODE ENTITY Task FIELDS ( status ) WITH
CORRESPONDING #( keys ) RESULT DATA(lt_task).
```

```
LOOP AT lt_task INTO DATA(ls_task) WHERE status <> 'O' AND status <> 'D'. APPEND VALUE #( %key =
ls_task-%key ) TO failed-task. APPEND VALUE #( %key = ls_task-%key %msg = new_message_with_text(
severity = if_abap_behv_message=>severity-error text = |Status { ls_task-status } is not a valid value| ) ) TO
reported-task. ENDLOOP. ENDMETHOD.
```

```
ENDCLASS. ``
```

注意宣告用的正是官方現行語法——**FOR DETERMINE ON SAVE / FOR VALIDATE ON SAVE**，跟舊課程 rap05 / rap06 被迫改用的 **obsolete FOR DETERMINATION / FOR VALIDATION** (On-Premise 1909 系統的剖析器只認得到這個較舊的版本) 形成直接對照——這是這個 **Cloud** 環境版本領先帶來的具體差異，不是巧合。

⚠️ 這個環境的 **MCP 工具限制**：**replace_string_in_abap_object** 對含有 **IMPORTING <param> FOR <something>** 這種語法 (RAP Handler Method 宣告的招牌寫法) 的內容，存檔一律失敗 (**An error occurred during the save operation**)，不分是不是真的 RAP Handler，只要是這個語法形狀就會存檔失敗；純 **METHODS xxx.** (無參數) 或一般 **IMPORTING xxx TYPE yyy** 都正常。這是這門課第一個確認的 **MCP 工具存檔限制** (另一個是 rc01 就記錄過的「建立新物件」全面故障)。Workaround：這類內容一律請使用者直接在 Eclipse 貼上 (Eclipse 原生存檔不受影響)，Claude 事後用 **get_abap_diagnostics / run_unit_tests** 驗證行為正確即可，不用執著於一定要用 MCP 工具寫入。

IN LOCAL MODE：Eclipse 編輯器主動跳出的提示，不是憑空猜的

上面兩個 Handler Method 內部呼叫 **MODIFY ENTITIES / READ ENTITIES** 操作自己的 Entity (改自己的欄位、包含 **field(readonly)** 欄位)，都明確加了 **IN LOCAL MODE**。這個線索不是查文件查到的，是 **Eclipse 編輯器** 自己主動跳出的提示：滑鼠移到 **MODIFY ENTITIES/READ ENTITIES** 那一行，會顯示：

When calling your own behavior implementation, statement variant "IN LOCAL MODE" (which skips permission checks) would improve performance and prevent incomplete results.

語法位置：**MODIFY ENTITIES OF <bdef> IN LOCAL MODE ENTITY ...** (**IN LOCAL MODE** 在 **OF <bdef>** 之後、**ENTITY** 之前)。查證官方文件 **ABAPMODIFY_ENTITY_ENTITIES** 確認語法位置正確。

一開始曾經懷疑是不是 **field(readonly)** 標記本身擋住了 Determination 對 **created_at/created_by** 的寫入 (舊課程 rap05/rap06 記錄過類似的 On-Premise 限制)，但實測拿掉 **readonly** 標記後、不加 **IN LOCAL MODE**，問題一樣存在——證實跟 **readonly** 無關，真正原因是 Handler Method 呼叫自己 Entity 的 EML 陳述式，本來就應該用 **IN LOCAL MODE** 跳過權限檢查、避免不完整的結果 (呼應 **authorization master(none)** 這個情境下更容易被忽略，因為「反正沒有權限檢查」的直覺會讓人誤以為不需要 **IN LOCAL MODE**)。

這一課的兩個 **Handler Method** 都要加 **IN LOCAL MODE**——**setCreationInfo** 的 **MODIFY ENTITIES**、**validateStatus** 的 **READ ENTITIES**，缺一個都會導致該方法內部的操作行為不完整 (實測記錄：只加第一個時，**CREATE** 測試會出現一個沒見過的 **FAILED_LATE** 衍生型別、驗證失敗；兩個都加上之後，兩個測試方法才雙雙通過，見下方執行結果)。

官方文件證實：**COMMIT ENTITIES** 失敗後 Transactional Buffer 不會自動清空——這一課最重要的發現

驗證 Validation 是否真的擋掉非法資料時，第一版測試邏輯是：CREATE 一筆 **status='X'** (不合法) 的資料 → **COMMIT ENTITIES** (預期 **FAILED** 有值) → 緊接著 **READ ENTITIES** 驗證這筆資料**沒有**被寫入資料庫。結果：**COMMIT** 確實正確回報失敗 (證實 Validation 真的有被呼叫、真的擋下了)，但緊接著的 **READ ENTITIES** 卻讀到了 1 筆——看起來像是「驗證失敗了，資料卻還是被存進去」。

先直接查資料庫排除假象——用 **execute_data_query** 下 SQL 直接查 **zrc01_task** 表：

```
SELECT task_id, description, status, created_at, created_by
FROM zrc01_task WHERE task_id = 'RC02TEST02'
```

結果是 **0 筆**——資料庫裡根本沒有這筆資料，**READ ENTITIES** 讀到的「1 筆」不是真的持久化資料，是別的東西。

查證官方文件 **ABENBDL_VALIDATIONS** / **ABAPROLLBACK_ENTITIES** 找到答案：

The operation fails and the complete content of the transactional buffer of the current RAP transaction is rejected. A commit to the database happens only if all data changes are consistent. One inconsistency leads to a rejection of all the content in the transactional buffer.

No further data changes are possible as long as the invalid instances aren't corrected. An invalid entity instance must either be corrected using an update operation, or the transactional buffer must be cleared using the EML statement **ROLLBACK ENTITIES**. **The reason for this is that the COMMIT ENTITIES statement aborts the save sequence in case of invalid instances and the transactional buffer is not emptied.**

關鍵字是「**the transactional buffer is not emptied**」：**COMMIT ENTITIES** 驗證失敗時，資料庫確實完全沒有寫入 (「整批拒絕」的語意是真的)，但緩衝區裡那筆「失敗」的資料不會自動清掉——所以緊接著的 **READ ENTITIES** (會先查緩衝區、緩衝區沒有才查資料庫) 讀到的其實是還卡在緩衝區裡、從未真正持久化的那筆資料，而不是資料庫裡真的有這筆資料。必須明確呼叫 **ROLLBACK ENTITIES** (沒有任何參數、也不能加 **OF <bdef>**) 才會清空緩衝區、讓 **READ ENTITIES** 之後查到的才是資料庫的真實狀態：

```
" ---- COMMIT: Validation 應該在這裡擋下來 ----
COMMIT ENTITIES
  RESPONSE OF zi_rc01_task
  FAILED DATA(ls_failed_commit)
  REPORTED DATA(ls_reported_commit).

cl_abap_unit_assert=>assert_not_initial( ls_failed_commit-task ).

" ---- 緩衝區清空，才能看到真實的資料庫狀態 ----
ROLLBACK ENTITIES.

READ ENTITIES OF zi_rc01_task
  ENTITY Task
  FIELDS ( task_id )
```

```
WITH VALUE #( ( %key-task_id = lv_id ) )  
RESULT DATA(lt_read).  
  
cl_abap_unit_assert=>assert_equals( act = lines( lt_read ) exp = 0 ).
```

這是一個很好的教訓：**COMMIT ENTITIES** 的 **FAILED/REPORTED** 告訴你「這次存檔有沒有成功」，但**不會**告訴你「緩衝區狀態恢復乾淨了沒」——這是兩件獨立的事。看到 **FAILED** 有值之後，如果你的程式邏輯打算繼續在同一個 RAP 交易裡做別的操作（不管是驗證用的 **READ ENTITIES**，還是接下來想嘗試別的 **CREATE**），都要先 **ROLLBACK ENTITIES** 把緩衝區清乾淨，不然後續行為都會被這筆「卡住」的失敗資料影響——官方文件第三條規則說得更直接：「**No further data changes are possible as long as the invalid instances aren't corrected**」，緩衝區裡有一筆沒處理的失敗資料，會擋住後續所有新的資料異動，直到你 **ROLLBACK ENTITIES** 或者把那筆資料改到合法為止。

ABAP Unit 執行結果

```
Unit Test Results for CLAS/I ZCL_RC02_TASK_TEST.main  
Status: ALL TESTS PASSED  
Total: 2 | Passed: 2 | Failed: 0  
  
[PASS] ZCL_RC02_TASK_TEST  
  [PASS] CREATE_INVALID_STATUS (0.060s)  
  [PASS] CREATE_UPDATE_DELETE (0.090s)
```

CREATE_UPDATE_DELETE (rc02 就有的測試，這一課擴充了驗證項目) 額外確認 **CREATE** 之後 **created_at/created_by** 兩個唯讀欄位真的被 **Determination** 自動填值 (**assert_not_initial**) ；
CREATE_INVALID_STATUS (這一課新增) 完整驗證 **Validation** 擋下非法 **status** 值、**COMMIT** 正確回報失敗、**ROLLBACK ENTITIES** 之後資料庫確實乾乾淨淨。

學習目標

- 能寫出完整 **strict(2)** 版本的 **Managed BDEF**：**managed implementation in class ... / authorization master(...) / determination ... on save { ... } / validation ... on save { ... }**
- 知道 **authorization master(none)** 的明確語意 (官方文件：這個值下不能實作任何權限 **Handler Method**)，以及為什麼這一課選它——隔離變因，專心示範 **Determination/Validation**
- 能講出 **Handler Method** 該寫在哪裡：全域類別 (**FOR BEHAVIOR OF**) 維持空殼，真正邏輯寫在繼承 **CL_ABAP_BEHAVIOR_HANDLER** 的 **Local Class** (**Local Types** 分頁)，知道直接寫在全域類別會被 **Eclipse** 擋下並給出明確錯誤訊息
- 能講出這個 **Cloud** 環境的 **Determination/Validation Handler** 宣告用官方現行語法 (**FOR DETERMINE ON SAVE / FOR VALIDATE ON SAVE**)，對照舊課程 rap05 / rap06 因為 **On-Premise** 系統版本落後被迫用 **obsolete** 語法 (**FOR DETERMINATION / FOR VALIDATION**)
- 知道 **Handler Method** 內部呼叫自己 **Entity** 的 **MODIFY ENTITIES/READ ENTITIES** 該加 **IN LOCAL MODE**，以及這個線索是 **Eclipse** 編輯器主動提示、不是憑空猜測
- 能講出這一課最重要的發現：**COMMIT ENTITIES** 驗證失敗後，**Transactional Buffer** 不會自動清空 (官方文件明文記載)，必須明確呼叫 **ROLLBACK ENTITIES** 才能讓後續的 **READ ENTITIES** 反映資料庫的真實狀態、也才能讓 **RAP** 交易恢復可以繼續操作的狀態

- 知道 `abap-remote-fs` 的 `replace_string_in_abap_object` 工具對 `IMPORTING xxx FOR yyy` 這種語法存檔會失敗，遇到時改請使用者在 Eclipse 貼上，Claude 事後驗證行為即可

物件清單

物件	名稱	型別
Managed Behavior Definition	<code>ZI_RC01_TASK</code> (跟 CDS View 同名，沿用 <code>rc02</code> ，這一課補上完整 <code>strict(2)</code> + Determination/Validation)	BDEF/BDO
Implementation Class (全域殼 + Local Types 的 <code>lhc_task</code>)	<code>ZBP_I_RC01_TASK</code>	CLAS/OC
ABAP Unit 測試類別	<code>ZCL_RC02_TASK_TEST</code> (沿用 <code>rc02</code> ，這一課擴充驗證項目 + 新增 <code>CREATE_INVALID_STATUS</code>)	CLAS/OC

套件：`ZRAPCLOUD` (沿用 `rc01/rc02`)。 `ZBP_I_RC01_TASK` 由使用者在 Eclipse 建立空殼 (Local Types 裡含 `IMPORTING ... FOR ...` 語法，MCP 工具存檔會失敗，內容由使用者直接在 Eclipse 貼上) ； BDEF / 測試類別由 Claude 用 MCP 讀寫、啟用、驗證。

驗證方式

- `get_abap_diagnostics` 確認 BDEF / `ZBP_I_RC01_TASK` / `ZCL_RC02_TASK_TEST` 三個物件都無語法錯誤
- `abap_activate` 全部回報 `Activation successful`
- `run_unit_tests` 對 `ZCL_RC02_TASK_TEST` 執行，`ALL TESTS PASSED` (見上方完整輸出)
- `execute_data_query` 直接查 `zrc01_task` 表，確認 Validation 拒絕的資料真的沒有進資料庫 (0 筆) ——不只依賴 EML 層面的驗證，額外做一次獨立於 RAP 框架之外的資料庫層驗證

思考題

- 這一課把 `validation validateStatus on save { field status; }` 的觸發條件設成 `field status` (只有 `status` 欄位變更才觸發)。如果改成 `create` (每次 `CREATE` 都觸發，不管有沒有動到 `status`)，行為上會有什麼差異？ (提示：想一想 `UPDATE` 操作如果完全沒碰 `status` 欄位，兩種寫法會不會都觸發驗證)
- `determination setCreationInfo on save { create; }` 如果改成 `on modify` (沒有這門課這樣寫，但語法上合法)，會在什麼時間點被呼叫？跟目前的 `on save` 有什麼差異？
- 官方文件的 `EML access 3` 提到：「如果一個不合法的實例沒有被修正或刪除，會擋住後續所有的資料異動」——這一課的測試每次都靠 `ROLLBACK ENTITIES` 清空緩衝區來繞過這個限制。如果不用 `ROLLBACK ENTITIES`，還有官方文件提到的另一種恢復方式，是什麼？ (提示：回顧上面「官方文件證實」那一節的引文)
- `authorization master(none)` 這一課用來隔離變因。如果之後要示範真正的權限檢查 (`authorization master(instance)`)，Handler Method 需要多實作哪一種方法？ (提示：可以先查 `ABAPHANDLER_METH_AUTH` 這份文件，這一課沒有深入，留給你自己探索)

答案

見 `zi_rc01_task.bdef.abap` (完整 `strict(2)` 版本)、`zbp_i_rc01_task.clas.abap` (全域殼 + Local Types `lhc_task` 完整內容)、`zcl_rc02_task_test.clas.abap` (`setup / create_update_delete /`

`create_invalid_status` 三個方法) 。SAP 端物件： `ZI_RC01_TASK` (BDEF) 、 `ZBP_I_RC01_TASK` (Implementation Class) 、 `ZCL_RC02_TASK_TEST` (ABAP Unit 測試類別) ，套件 `ZRAPCLOUD` ，
`run_unit_tests` 執行結果： `ALL TESTS PASSED` 。